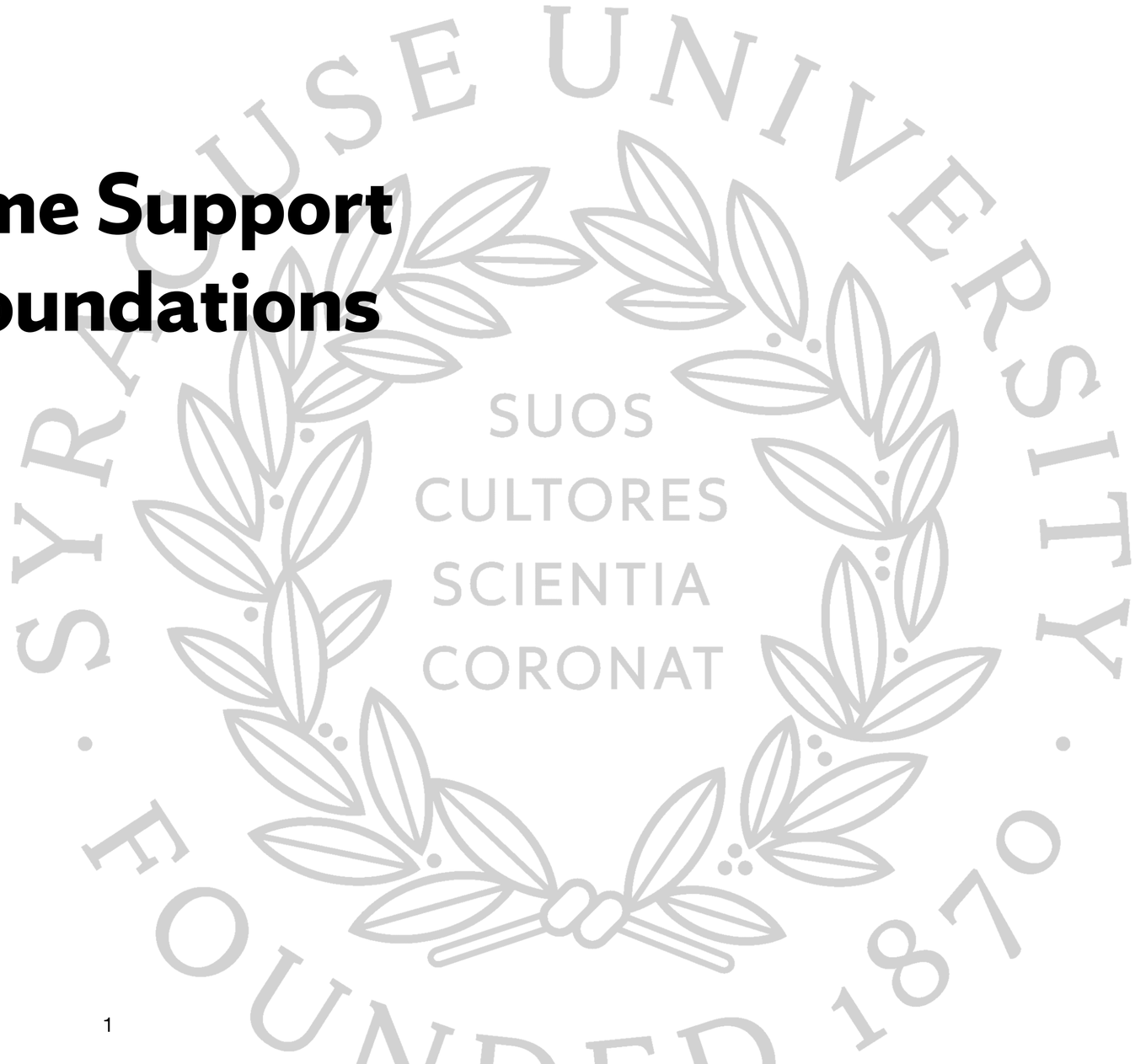
Type systems: Runtime Support and Mathematical Foundations

Kristopher Micinski

Syracuse University

CIS531, Fall 2025



Why Use Type Systems

- Two major uses of type systems in programming languages:
  - **Avoid unnecessary overhead to ensure safety**
    - To be safe, need to check certain structural assumptions
    - E.g., to do a safe vector reference, I need to know it's **actually a vector**
      - And not just a random i64!
    - Not checking these assumptions = major security vulnerability, segfault, etc.
    - Can eliminate costly dynamic checks by statically proving via type system
  - **Ensure mathematically-pleasing properties**
    - We know the program will never trigger an exception / deref NULL
    - We can prove that the procedure will always return a sorted list
    - Encode end-to-end system correctness via type system

Runtime typing

- Among the kinds of errors a program can face, we define **type errors**: errors which relate to structural mismatches between the expected type of a value and its runtime type.
  - E.g., ((lambda (x y z) 5) is a type error

- Racket isn't **statically** typed, but it still has a notion of **runtime** types
  - In dynamic typing, every value is tagged with a dynamic / runtime type

Why runtime type tags?

- Consider the following program:

```
(define (f v)
  (vector-ref v 2))

(f (vector 1 2 3))
(f 23)
```

- First call to f works fine, but second call leads to memory safety issue!

Spatial / Temporal Safety, Memory Errors

- We also talk about **memory safety**, which is the idea that our access to memory ensures…:
  - (a) all memory accessed lies within the bounds of a segment which has been allocated to the program. This is **spatial safety**
  - (b) all memory accessed has not been freed, either explicitly (e.g., by a call to free/delete) or implicitly (e.g., stack-allocated data is no longer valid after the function's return). This is **temporal safety**

- Memory safety generally factors into these two categories
  - Which is stack overflow? Use after free?

Spatial / Temporal Memory Safety Violations:

Which is which?

```
int main(void) {
    int a[4] = {0};
    for (int i = 0; i <= 4; i++)
        a[i] = i;
    printf("%d\n", a[0]);
}
```

```
int *bad(void) {
    int x = 7;
    return &x;
}
int main(void) {
    int *p = bad();
    return *p;
}
```

Type errors

- We will discuss both, but compilers generally concern themselves with the first bullet point
  - To what degree do the types actually inform how we compile…?
  - So far, in our compilers, we don't use type information
    - Assume program correctly typed—type checking a separate phase

- Among the kinds of errors a program can face, we define **type errors**: errors which relate to structural mismatches between the expected type of a value and its runtime type (need to define "type" to be coherent)
  - E.g., ((lambda (x y z) 5) is a type error

- Racket isn't **statically** typed, but it still has a notion of **runtime** types

Implementing Dynamic Typing…

- Basic issue: in dynamic typing **every object must be typed**
  - And worse: ***whenever*** we interact with **any** object, we must first check its dynamic type
  - This will, in general, lead to a potentially massive amount of overhead
- So the question is… how do we add runtime type tags to every object?

Approach 1: Box Everything

- **No** unboxed data. Your runtime system is the only place objects created
  - All runtime values have some known (by you) format
  - Disallow users creating pointers themselves
    - I.e., work in terms of ***references*** rather than *pointers*
      - (Pointers support arithmetic, references are opaque)
  - If you allow random objects with unknown formats (e.g., untrusted dynamic libraries), then all bets are off
    - FFI is tricky for this reason, need assumptions on layout, etc.

- 👍 — easier to ensure spatial safety, can check shapes dynamically
- 👎 — more indirection, can't do direct ops on (say) integers

Example: CPython's value representation

In CPython, **every** object is tagged with with a reference count and type information

```
#define PyObject_HEAD
    Py_ssize_t ob_refcnt;
    struct _typeobject *ob_type;
```

```
// longs in CPython
typedef struct {
    PyObject_HEAD
    long ob_ival;
} PyLongObject;

// i.e.,
typedef struct {
    Py_ssize_t ob_refcnt;
    struct _typeobject *ob_type;
    long ob_ival;
} PyLongObject;
```

Example: Glasgow Haskell Compiler's Runtime...

GHC ***does*** perform unboxing aggressively, this is the used for boxed types...

(Tries to keep ints in registers when possible...)

```
typedef struct {
   const StgInfoTable* info;
#if defined(PROFILING)
    StgProfHeader         prof;
#endif
} StgHeader;
```

```
// The "standard" part of an info table.  Every info table has this bit.
typedef struct StgInfoTable_ {

#if !defined(TABLES_NEXT_TO_CODE)
    StgFunPtr       entry;      /* pointer to the entry code */
#endif

#if defined(PROFILING)
    StgProfInfo     prof;
#endif

    StgClosureInfo  layout;     /* closure layout info (one word) */

    StgHalfWord     type;       /* closure type */
    StgSRTField     srt;
       /* In a CONSTR:
            - the zero-based constructor tag
          In a FUN/THUNK
            - if USE_INLINE_SRT_FIELD
              - offset to the SRT (or zero if no SRT)
            - otherwise
              - non-zero if there is an SRT, offset is in srt_offset
       */

#if defined(TABLES_NEXT_TO_CODE)
    StgCode         code[];
#endif
} *StgInfoTablePtr; // StgInfoTable defined in rts/Types.h
```

Approach 2: Just Assume Programmer Correct

- What C does: type system used to communicate **structural / memory-layout related concerns**
  - Type system used for static layout (e.g., sizeof), informs compilation
- Programmer can always create void* pointers, do pointer arithmetic
  - Allows ultimate flexibility, but at the cost of safety
  - If the actual type isn't the type assumed by the program, the program could totally corrupt memory (e.g., write past bounds of buffer)

- Interesting somewhat-related bug (see history / discussion):
  - https://issues.chromium.org/issues/382005099

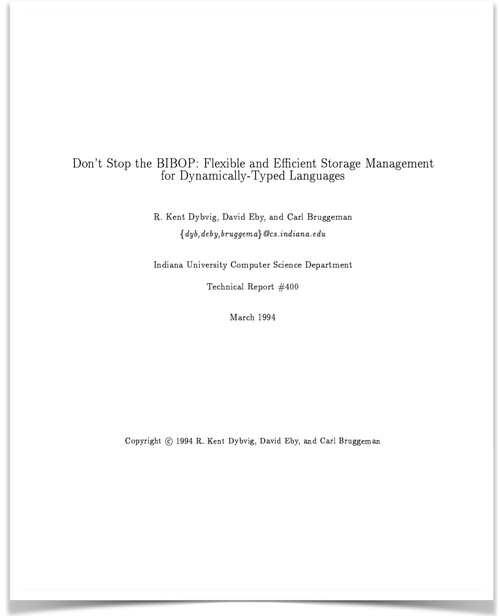
Approach 3: Big Bag of Pages (BIBOP)

- Store objects which share the same descriptor (roughly: type decl) in the ***same page of memory*** (page is a large chunk)
  - Most significant bits of the pointer determine type
- Headerless, page-typed layout
  - Runtime and generated code must agree on runtime representation
- Page metadata specifies object size (consulted by GC)

```
// Pair (cons cell): lives on a PAIR page
typedef struct {
  scm_obj car;
  scm_obj cdr;
} scm_pair;

// Flonum (IEEE double): lives on a FLONUM page
typedef struct {
  double val;
} scm_flonum;
```

```
// Vector (variable-sized): lives on a VECTOR page
typedef struct {
  size_t len;
  scm_obj elems[];   // len elements follow
} scm_vector;
```



Don't Stop the BIBOP: Flexible and Efficient Storage Management for Dynamically-Typed Languages

R. Kent Dybvig, David Eby, and Carl Bruggeman
{dyb,deby,bruggema}@cs.indiana.edu

Indiana University Computer Science Department

Technical Report #400

March 1994

The system described in this paper, therefore, makes use of the *Big Bag of Pages* (BIBOP) typing mechanism, which views memory as a group of equal-sized segments with an associated type table mapping segment numbers to types [20]. BIBOP typing has fallen into disfavor in recent years because newer type systems provide faster allocation and type manipulation and because of percevied problems with the handling of large objects. We have found, however, that these problems can be eliminated through the use of a hybrid typing system.

In our system, fast primary typing is provided by associating tags with pointers to objects (for most common types) or with the objects themselves. Secondary typing, or *metatyping*, is provided by the BIBOP type table. Metatypes allow the system to mark segments according to certain important characteristics. In particular, segments are marked as "holes" if they are contained within the heap but are not logically part of the heap, and they are marked "executable" if they contain executable code. Segments are also marked according to how the objects within them must be handled by the garbage collector, *i.e.*, objects containing no pointers to be swept, objects containing only pointers, and objects such as stacks which require customized sweeping are all kept in different kinds of segments. The generation to which each segment belongs is also recorded as part of a segment's metatype. Immutable objects can be separated from mutable objects to improve virtual memory behavior and to reduce the number of areas for which the generational garbage collector must look for pointers from older to younger generations.

The storage management system described in this paper has been implemented and is in use as part of *Chez Scheme*, a high-performance implementation of Scheme [8]. Although hybrid typing mechanisms and segmented memory models are not entirely new, we believe we are the first to implement a segmented memory model using an extensive set of metatypes with fast inline allocation. Also, many of the purposes for which we use metatypes appear to be unique, including separating code from data and separating mutable from immutable data. Our mechanisms for collecting stacks and code objects, which both contain a mix of pointer and nonpointer data, are also new[2] (see Section 5.3). We also describe a variant of card marking [19, 25] for remembering pointers from older generations to newer ones. This variant allows us to perform the marking inline while maintaining a record of the youngest generation involved, reducing card sweeping overhead when more than two generations are used. Many of the other features and derived benefits we discuss, *e.g.*, how our storage manager coexists with other languages' run-time systems, are likely not new but either have not been well documented in the literature or have not previously been implemented.

Approach 4: Runtime Pointer Tagging

- Fairly common: use lower 3 bits of pointer to store some metadata
  - Could be GC related (marked, etc.), could be type info, etc.
- E.g., OCaml's 63-bit immediate integers:
  - Every 64-bit value is **either** a pointer to a heap-allocated value ***or*** a 63-bit immediate integer
    - (You lose one bit for the single choice—worth it for non-standard?)
    - Can always work with boxed ints, or any other boxed numeric type
- Examples of famous implementations using some pointer tagging:
  - v8 (JavaScript engine) — small integer (SMI) vs. non-small integer
  - Apple's ObjC runtime — NSNumber, NSDate (low bits store type)
  - CRuby — tagging for immediates, heap objects for everything else

- ***Fundamental choice: is the language type / memory safe?***

- If NO: all bets are off, types are generally only to inform how memory layout / access should occur
  - Even if types not used for safety, still needed for some purposes (struct layout, etc.)
  - E.g., in our language: at least want to know you're using a vector
  - But always an "escape hatch:" C allows ***downcasting***
    - Can always downcast void* to any type — breaks type soundness

- If YES: need some combination of static type system / runtime overhead to ensure type / memory safety
  - If implemented naively, this means (potentially lots of) runtime overhead due to repeated checks
    - Compiler / runtime can often optimize runtime checks
      - But many still do not (CPython is ***very slow!!!***)
    - I recommend looking up "Zero-cost abstractions"

A simple extension of our class' language...

- Most languages / runtimes use a combination of typing and runtime representation to strike the desired performance balance
- In our case, we have decided to have two types: vectors and unboxed integers
  - Could implement a static type system to ensure type correctness
    - Not completely trivial, once we have functions / application
- Could also implement ***runtime tagging*** (EoC book — Chapter 8)
  - Every 64-bit value encoded using a tag
    - Int, Bool, Vetor, Function, or Void
  - Compile to include ***casts*** which box/unbox

$$
\begin{aligned}
tagof(\texttt{Integer}) &= 001 \\
tagof(\texttt{Boolean}) &= 100 \\
tagof((\texttt{Vector}\ldots)) &= 010 \\
tagof((\ldots\texttt{->}\ldots)) &= 011 \\
tagof(\texttt{Void}) &= 101
\end{aligned}
$$

type	::=	... | `Any`
op	::=	... | `any-vector-length` | `any-vector-ref` | `any-vector-set!`
	|	`boolean?` | `integer?` | `vector?` | `procedure?` | `void?`
exp	::=	... | `(Prim` *op* `(`*exp*`...))`
	|	`(Inject` *exp ftype*`)` | `(Project` *exp ftype*`)`
def	::=	`(Def` *var* `([`*var*`:`*type*`]...)` *type* `'()` *exp*`)`
$R_{\texttt{Any}}$	::=	`(ProgramDefsExp '()` `(`*def*`...)` *exp*`)`

Figure 8.5: The abstract syntax of $R_{\texttt{Any}}$, extending R_{λ} (Figure 7.4).

Static Type Systems

A **static type system** associates each source code fragment with a given **type**: an approximation of how it will behave

Type systems include **rules**, or **judgements** that tells us how we compositionally build types for larger fragments from smaller fragments

The goal of a type system is to rule out / reject programs that would exhibit run time type errors!

Which of the following should be allowed to run?

```
(λ (g) (g 5))
```

```
((λ (f x) (f x)) g)
```

```
(λ (f) (+ (f 1) (if (= 0 (string-length (f 1))) 1 0)))
```

Which of the following should be allowed to run?

```
(λ (g) (g 5))
```

Nothing **obviously** wrong, in absence of more knowledge about g

```
((λ (f x) (f x)) g)
```

You **cannot** call this lambda, it will ***necessarily*** result in an error

```
(λ (f) (+ (f 1) (if (= 0 (string-length (f 1))) 1 0)))
```

Can't work if `f` is pure (not stateful): `(f 1)` can't return a number and a string

Type systems will give us a formal (in the sense of having a form we can write down) description of an expression's runtime form

A type is a rough approximation of the expression's behavior. For example, the type `int` might represent the type of all integers, while the type `int -> int` would be the functions from values of type `int` to values of type `int`

(Preview of where we're going)

We'll be able to use a type system to be able to deduce that, because x and y are passed to +, they **must** be ints (+ constraints its arguments to be ints)

```
;; OCaml, not Racket
((fun x y -> (x + y)) 1 2);;
```

This process called **type inference**, and is common in modern languages (Examples include Rust, TypeScript, Haskell, OCaml …)

You can think of type inference as an *always correct CoPilot*, but the correctness also means expressivity is limited only to properties about which the type system is designed to reason.

A type inference system means that you write as many annotations as you want—the compiler figures out what you mean and tells you when it hits an inconsistency.

Question: is this code okay?

```
# (fun x f -> (if (f 3) then ((f x) + 5) else 8));;
```

A type inference system means that you write as many annotations as you want—the compiler figures out what you mean and tells you when it hits an inconsistency.

```
# (fun x f -> (if (f 3) then ((f x) + 5) else 8));;
Error: This expression has type bool but an
expression was expected of type
         int
```

In type theory, a subexpression has a **type** when there exists some **proof** according to a formally-defined typing derivation.

You will learn how to write proofs for typing derivations in the Simply-Typed Lambda Calculus, a small core of a type system for a functional language.

Simply-Typed λ-calculus

STLC is a restriction of the untyped λ-calculus
(It is a restriction in the sense that not all terms are well-typed.)

Expressions in STLC, assuming t is a type (we'll show this soon):

```
e ::= (lambda (x : t) e)
    | (e e)
    | (prim e e)
    | x
    | n
    | (e : t)

prim ::= + | * | …
```

All lambdas ***must*** be annotated with their type

Optionally, any subexpression may be ***annotated*** with a type

```
;; Expressions are ifarith, with several special builtins
(define (expr? e)
  (match e
    ;; Variables
    [(? symbol? x) #t]
    ;; Literals
    [(? bool-lit? b) #t]
    [(? int-lit? i) #t]
    ;; Applications
    [`(,(? expr? e0) ,(? expr? e1)) #t]
    ;; Annotated expressions
    [`(,(? expr? e) : ,(? type? t)) #t]
    ;; Anotated lambdas
    [`(lambda (,(? symbol? x) : ,(? type? t)) ,(? expr? e)) #t]))
```

The ***simply typed*** lambda calculus is a type system built on top of a small kernel of the lambda calculus

Crucially, STLC is *less expressive* than the lambda calculus (e.g., we cannot type Ω, Y, or U!)

In practice, STLC's restrictions make it unsuitable for serious programming—but it is the basis for many modern type systems in real languages (e.g., OCaml, Rust, Swift, Haskell, …)

Terms ***inhabit*** types
(via the typing judgement)

Term Syntax

```
e ::= (lambda (x : t) e)
    | (e e)
    | (prim e e)
    | x
    | n
    | (e : t)

prim ::= + | * | …
```

Type Syntax

```
t ::= int
    | bool
    | t -> t
```

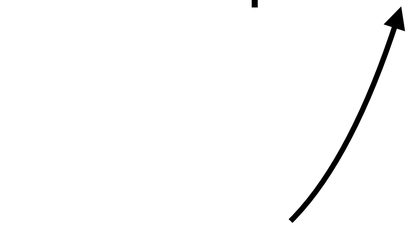
Term Syntax

```
e ::= (lambda (x : t) e)
    | (e e)
    | (prim e e)
    | x
    | n
    | (e : t)

prim ::= + | * | …
```

Type Syntax

```
t ::= int
    | bool
    | t -> t
```



Function Types

Term Syntax

```
e ::= (lambda (x : t) e)
    | (e e)
    | (prim e e)
    | x
    | n
    | (e : t)

prim ::= + | * | …
```

Type Syntax

```
t ::= int
    | bool
    | t -> t
```

Examples…

```
bool -> int

int -> (int -> int)

int -> int

(int -> int) -> int

(bool -> (int -> bool)) -> int
```

- Type checking happens hierarchically
- Literals (0, #f) have their obvious types
- More complex forms (lambda, apply) require us to type subexpressions

For example, let's say we have this lambda, which we want to type check:

$$\Big(\lambda(x : \text{int})\ \big(\mathit{if}\ (x = 0)\ x\ (+\ x\ 1)\big)\Big)$$

First we see the input type is int. Assuming x is int, we type check the body (an if). We see both sides of the if result in a number, so we know the lambda's output is also a number.

Thus, the type is `int -> int`

The fact that lambdas must be annotated with a type makes typing easy: parameters are the only true source of non-local control in the lambda calculus, and represent the only ambiguity in type checking

$$\left(\lambda(x : \text{int} \to \text{int})\ \left(\mathit{if}\ \#\text{f}\ (x\ 5)\ (x\ 8)\right)\right)$$

We know both the input and output type

Bad thought experiment

$$\left(if\ \#\mathrm{f}\ (x\ 5)\ (x\ 8)\right)$$

Let's say x is the Racket lambda:

```
(λ (x) (if (< x 6) #t 5))
```

Now, when x is less than 6, we return something of type **`bool`**; but otherwise, we return something of type **`int`**.

$$\left(+\ 3\ \left(if\ \#\mathrm{f}\ (x\ 5)\ (x\ 8)\right)\right)$$

In this case, the + operation works as long as **`(x 8)`** returns a **`int`**, but what if **`(x 8)`** returns a bool?

We could write a **union type**, but STLC will make things simpler: the true/false branch **must** have the same type.

A few examples...

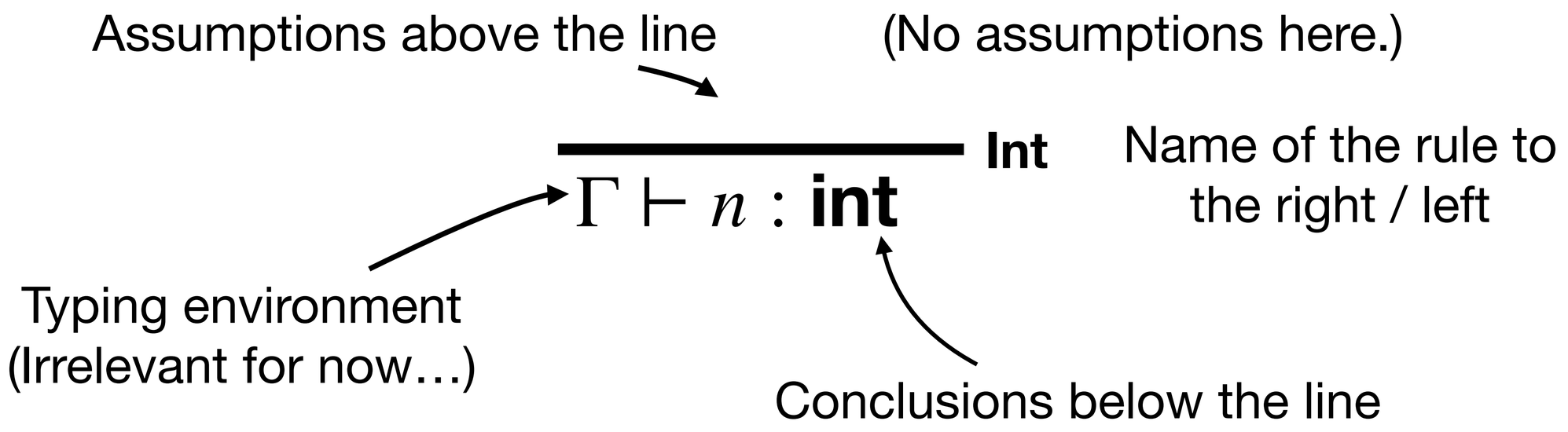
$$\left(\lambda(x : \text{int})\ (\lambda\ (y : \text{bool})\ y)\right) : \text{int} \to \text{bool} \to \text{bool}$$

$$\left(\lambda(x : \text{int} \to \text{int})\ (x\ 5)\right) : (\text{int} \to \text{int}) \to \text{num}$$

$$\left(\lambda(x : \text{int} \to \text{int})\ \left(\mathit{if}\ \#\text{f}\ (x\ 5)\ (x\ 8)\right)\right) : (\text{int} \to \text{int}) \to \text{int}$$

STLC Typing Rules

Type rules are written in ***natural-deduction*** style



The rule reads “in any typing environment Γ, we may conclude the literal number n has type int”

$$\frac{}{\Gamma \vdash n : \textbf{int}}\ \textbf{Int}$$

Variable Lookup

We assume a **typing environment** which maps variables to their types

$$\frac{\Gamma(x) = t}{\Gamma \vdash x : t}\ \textbf{Var}$$

If x maps to type t in Γ, we may conclude that x has type t under the type environment Γ

Exercise: using the **Var** rule, complete the proof

$$\frac{}{\{x \mapsto (\textbf{int} \to \textbf{int}), y \mapsto \textbf{bool}\} \vdash x : ???}$$

$$\frac{\Gamma(x) = t}{\Gamma \vdash x : t}\ \textbf{Var}$$

Solution

$$\frac{\{x \mapsto (\textbf{int} \to \textbf{int}), y \mapsto \textbf{bool}\}(x) = \textbf{int} \to \textbf{int}}{\{x \mapsto (\textbf{int} \to \textbf{int}), y \mapsto \textbf{bool}\} \vdash x : (\textbf{int} \to \textbf{int})}\ \textbf{Var}$$

$$\frac{\Gamma(x) = t}{\Gamma \vdash x : t}\ \textbf{Var}$$

Typing Functions

If, assuming x has type x, you can conclude the body e has type t', then the whole lambda has type t → t'

$$\frac{\Gamma[x \mapsto t] \vdash e : t'}{\Gamma \vdash (\lambda\,(x : t)\ e) : t \to t'}\ \textbf{Lam}$$

If, assuming x has type x, you can conclude the body e has type t', then the whole lambda has type t → t'

$$\frac{\Gamma[x \mapsto t] \vdash e : t'}{\Gamma \vdash (\lambda\,(x : t)\ e) : t \to t'}\ \textbf{Lam}$$

Notice: if we didn't have type t here, we would have to ***guess***, which could be quite hard. We will have to do this when we move to allow *type inference*

$$\frac{\Gamma[x \mapsto t] \vdash e : t'}{\Gamma \vdash (\lambda\,(x : t)\ e) : t \to t'}\ \textbf{Lam}$$

Example: let's use the Lam rule to ascertain the type of the following expression.

$$\frac{}{\texttt{(lambda (x : int) 1)}}$$

$$\frac{\Gamma[x \mapsto t] \vdash e : t'}{\Gamma \vdash (\lambda\,(x : t)\ e) : t \to t'}\ \textbf{Lam}$$

Start with the empty environment (since this term is closed)

$$\frac{}{\Gamma = \{\} \vdash \texttt{(lambda (x : int) 1)} : ? \to ?}$$

$$\frac{\Gamma[x \mapsto t] \vdash e : t'}{\Gamma \vdash (\lambda\,(x : t)\ e) : t \to t'}\ \textbf{Lam}$$

$$\Gamma = \{\} \vdash \frac{}{\texttt{(lambda (x : int) 1)} : t \to t'}$$

We **suppose** there are two types, t and t', which will make the derivation work.

Because x is tagged, it must be **int**

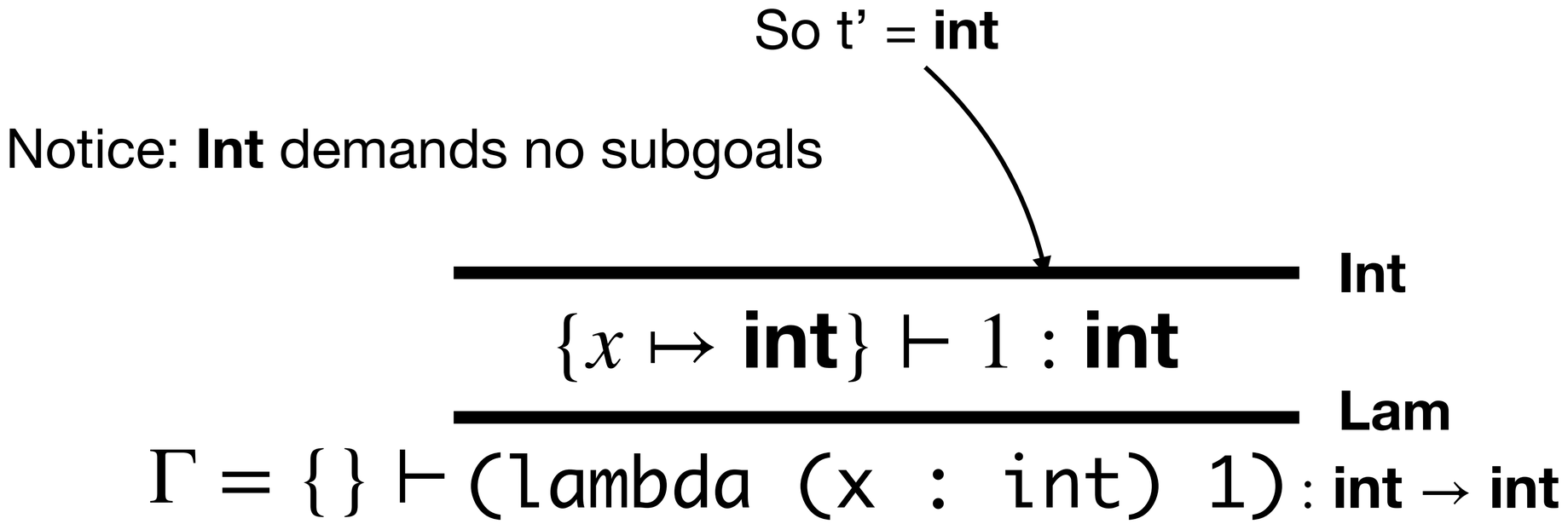
$$\frac{\{x \mapsto \textbf{int}\} \vdash 1 : t'}{\Gamma = \{\} \vdash \texttt{(lambda (x : int) 1)} : \textbf{int} \to t'}$$

We **suppose** there are two types, t and t', which will make the derivation work.

The **Int** rule allows us to conclude 1 : **int**

$$\dfrac{\dfrac{}{\{x \mapsto \textbf{int}\} \vdash 1 : t'}}{\Gamma = \{\} \vdash \texttt{(lambda (x : int) 1)} : \textbf{int} \to t'}\ \textbf{Lam}$$

We **suppose** there are two types, t and t', which will make the derivation work.


So t' = int
Notice: Int demands no subgoals
Int
$\{x \mapsto \textbf{int}\} \vdash 1 : \textbf{int}$
Lam
$\Gamma = \{\} \vdash$ (lambda (x : int) 1) : int → int

Function Application

$$\frac{\Gamma \vdash e : t \to t' \qquad \Gamma \vdash e' : t}{\Gamma \vdash (e\ e') : t'}\ \textbf{App}$$

Function Application

If (under Gamma), e has type `t -> t'`

And e' (its argument) has type t

$$\frac{\Gamma \vdash e : t \to t' \qquad \Gamma \vdash e' : t}{\Gamma \vdash (e\ e') : t'}\ \textbf{App}$$

Then the application of e to e' results in a `t'`

Our type system so far…

$$\frac{}{\Gamma \vdash n : \textbf{int}}\ \textbf{Int} \qquad \frac{}{\Gamma \vdash \textbf{\#t} : \textbf{bool}}\ \textbf{True}\ \text{(Also \textbf{False})} \qquad \frac{\Gamma(x) = t}{\Gamma \vdash x : t}\ \textbf{Var}$$

$$\frac{\Gamma \vdash e : t \to t' \quad \Gamma \vdash e' : t}{\Gamma \vdash (e\ e') : t'}\ \textbf{App}$$

$$\frac{\Gamma[x \mapsto t] \vdash e : t'}{\Gamma \vdash (\lambda\,(x : t)\ e) : t \to t'}\ \textbf{Lam}$$

Almost everything! What about builtins?

- Almost everything! What about builtins?
- A few ways to handle this:
  - Add ***pairs*** to our language
    - We'll see this next time

$$\Gamma_\iota = \{ + : (\textbf{num} \times \textbf{num}) \to \textbf{num}, \ldots \}$$

- Or, we could assume that primitives are simply curried—in that case we would have, e.g., ((+ 1) 2) and then…

$$\Gamma_\iota = \{ + : \textbf{num} \to (\textbf{num} \to \textbf{num}), \ldots \}$$

Two possibilities (pairs/currying)

```
e ::= (lambda (x : t) e)
    | (e e)
    | (prim (e, e)) ; pairs
    | ((prim e) e)  ; curry
    | x
    | n
    | (e : t)

prim ::= + | * | …
```

Practice Derivations

Write derivations of the following expressions…

```
((λ (x : int) x) 1)
```

$$\frac{}{\Gamma \vdash n : \textbf{int}}\ \textbf{Int} \qquad \frac{x \mapsto t \in \Gamma}{\Gamma \vdash x : t}\ \textbf{Var}$$

$$\frac{\Gamma \vdash e : t \to t' \quad \Gamma \vdash e' : t}{\Gamma \vdash (e\ e') : t'}\ \textbf{App}$$

$$\frac{\Gamma, \{x \mapsto t\} \vdash e : t'}{\Gamma \vdash (\lambda\,(x : t)\ e) : t \to t'}\ \textbf{Lam}$$

```
((λ (x : int) x) 1)
```

$$\dfrac{\dfrac{\dfrac{}{\{x \mapsto \textbf{int}\} \vdash x : \textbf{int}}\,\textbf{Var}}{\{\} \vdash (\lambda\,(x : \textbf{int})\ x) : \textbf{int} \to \textbf{int}}\,\textbf{Lam} \qquad \dfrac{}{\{\} \vdash 1 : \textbf{int}}\,\textbf{Int}}{\{\} \vdash \Big((\lambda\,(x : \textbf{int})\ x)\ 1\Big) : \textbf{int}}\,\textbf{App}$$

```
((λ (f : int -> int) (f 1)) (λ (x : int) x))
```

$$\frac{}{\Gamma \vdash n : \textbf{int}}\ \textbf{Int} \qquad \frac{x \mapsto t \in \Gamma}{\Gamma \vdash x : t}\ \textbf{Var}$$

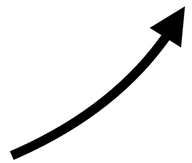
$$\frac{\Gamma \vdash e : t \to t' \quad \Gamma \vdash e' : t}{\Gamma \vdash (e\ e') : t'}\ \textbf{App}$$

$$\frac{\Gamma, \{x \mapsto t\} \vdash e : t'}{\Gamma \vdash (\lambda\,(x : t)\ e) : t \to t'}\ \textbf{Lam}$$

Typability in STLC

Not all terms can be given types…

```
(λ (f : int -> int) (f f))
```



It is impossible to write a derivation for the above term!

f is `int->int` but would **need** to be `int`!

Typability

Not all terms can be given types…

```
((λ (f) (f f))
 (λ (f) (f f)))
```

It is **impossible** to write a derivation for Ω!

Consider what would happen if `f` were:

- `int -> int`
- `(int -> int) -> int`

Always just out of reach…

Type **Checking**

Type checking asks: given this fully-typed term, is the type checking done correctly?

```
((λ (x:int) x:int) : int -> int)
```

In practice, as long as we annotate arguments (of λs) with specific types, we can elide the types of variables, literals, and applications

The “simply typed” nature of STLC means that type inference is very simple…

Exercise

For each of the following expressions, do they type check?
I.e., is it possible to construct a typing derivation for them?
If so, what is the type of the expression?

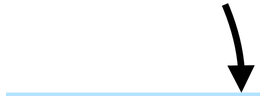
```
(λ (f : int -> int -> int) (((f 2) 3) 4))
```

```
((λ (f : int -> int) f) (λ (x:int) (λ (x:int) x)))
```

Solution

***Neither** type checks.*

This subexpression results in **int**, which cannot be applied.



```
(λ (f : int -> int -> int) (((f 2) 3) 4))
```

```
((λ (f : int -> int) f) (λ (x:int) (λ (x:int) x)))
```

Solution

Neither *type checks.*

```
(λ (f : int -> int -> int) (((f 2) 3) 4))
```

```
((λ (f : int -> int) f) (λ (x:int) (λ (x:int) x)))
```

This binder ***demands*** its argument is of type `int -> int`,
but its argument is ***really*** of type `int -> int -> int`

In the case of fully-annotated STLC, we never have to *guess* a type

In STLC, type *inference* is no harder than type *checking*

Our type checker will be **syntax-directed**

Next lecture, we will look at type inference for **un-annotated** STLC

- This will require generating, and then solving, constraints

The basic approach is to observe that each of the rules applies to a different *form*

$$\frac{}{\Gamma \vdash n : \textbf{int}}\ \textbf{Int} \qquad \frac{\Gamma(x) = t}{\Gamma \vdash x : t}\ \textbf{Var}$$

For example, if we hit *any* application expression (e e'), we know that we *have* to use the **App** rule

$$\frac{\Gamma \vdash e : t \to t' \quad \Gamma \vdash e' : t}{\Gamma \vdash (e\ e') : t'}\ \textbf{App}$$

Thus, we write our type checker as a structurally-recursive function over the input expression.

$$\frac{\Gamma[x \mapsto t] \vdash e : t'}{\Gamma \vdash (\lambda\,(x : t)\ e) : t \to t'}\ \textbf{Lam}$$

```
;; Synthesize a type for e in the environment env
;; Returns a type
(define (synthesize-type env e)
  (match e
    ;; Literals
    [(? integer? i) 'int]
    [(? boolean? b) 'bool]
```

$$\frac{}{\Gamma \vdash n : \textbf{int}}\ \textbf{Int}$$

Recognizing literals is easy

```
;; Synthesize a type for e in the environment env
;; Returns a type
(define (synthesize-type env e)
  (match e
    ;; Literals
    [(? integer? i) 'int]
    [(? boolean? b) 'bool]
    ;; Look up a type variable in an environment
    [(? symbol? x) (hash-ref env x)]
```

$$\frac{\Gamma(x) = t}{\Gamma \vdash x : t}$$ **Var**

```
;; Synthesize a type for e in the environment env
;; Returns a type
(define (synthesize-type env e)
  (match e
    ;; Literals
    [(? integer? i) 'int]
    [(? boolean? b) 'bool]
    ;; Look up a type variable in an environment
    [(? symbol? x) (hash-ref env x)]
    ;; Lambda w/ annotation
    [`(lambda (,x : ,A) ,e)
     `(,A -> ,(synthesize-type (hash-set env x A) e))]
```

$$\frac{\Gamma[x \mapsto t] \vdash e : t'}{\Gamma \vdash (\lambda\,(x : t)\ e) : t \to t'}\ \textbf{Lam}$$

```
;; Synthesize a type for e in the environment env
;; Returns a type
(define (synthesize-type env e)
  (match e
    ;; Literals
    [(? integer? i) 'int]
    [(? boolean? b) 'bool]
    ;; Look up a type variable in an environment
    [(? symbol? x) (hash-ref env x)]
    ;; Lambda w/ annotation
    [`(lambda (,x : ,A) ,e)
     `(,A -> ,(synthesize-type (hash-set env x A) e))]
    ;; Arbitrary expression
    [`(,e : ,t) (let ([e-t (synthesize-type env e)])
                  (if (equal? e-t t)
                    t
                    (error (format "types ~a and ~a are different" e-t t))))]
```

We haven't written this rule yet—but notice how the t's are implicitly unified (i.e., asserted to be the same) in the rule

$$\frac{\Gamma \vdash e : t}{\Gamma \vdash (e : t) : t}\ \textbf{Chk}$$

```
;; Synthesize a type for e in the environment env
;; Returns a type
(define (synthesize-type env e)
  (match e
    ;; Literals
    [(? integer? i) 'int]
    [(? boolean? b) 'bool]
    ;; Look up a type variable in an environment
    [(? symbol? x) (hash-ref env x)]
    ;; Lambda w/ annotation
    [`(lambda (,x : ,A) ,e)
     `(,A -> ,(synthesize-type (hash-set env x A) e))]
    ;; Arbitrary expression
    [`(,e : ,t) (let ([e-t (synthesize-type env e)])
                  (if (equal? e-t t)
                    t
                    (error (format "types ~a and ~a are different" e-t t))))]
    ;; Application
    [`(,e1 ,e2)
     (match (synthesize-type env e1)
       [`(,A -> ,B)
        (let ([t-2 (synthesize-type env e2)])
          (if (equal? t-2 A)
            B
            (error (format "types ~a and ~a are different" A t-2))))])]))
```

$$\frac{\Gamma \vdash e : t \to t' \quad \Gamma \vdash e' : t}{\Gamma \vdash (e\ e') : t'}\ \textbf{App}$$

The Curry-Howard Isomorphism

The Curry-Howard Isomorphism is a name given to the idea that every **typed lambda calculus** expression is a computational interpretation of a **theorem** in a suitable constructive logic.

For STLC: every well-typed term in STLC is a **theorem** in intuitionistic propositional logic (STLC ~= IPL).

So far, we have discussed four rules in STLC: Var, Const, App, and Lam

These rules ***exactly mirror*** corresponding rules in IPL

The **Var** rule corresponds to the **Assumption** rule
In IPL, Γ is a **set** of propositions (assumed true)
In STLC, Γ is a **map** from type variables to their types

$$\frac{x \mapsto t \in \Gamma}{\Gamma \vdash x : t}\ \textbf{Var}$$

$$\textbf{Assumption}\ \frac{}{\Gamma, P \vdash P}$$

Γ : **Var** → **Type**

Γ : **Set**(**Proposition**)

$$\frac{x \mapsto t \in \Gamma}{\Gamma \vdash x : t}\ \textbf{Var}$$

$$\textbf{Assumption}\ \frac{}{\Gamma, P \vdash P}$$

$$\frac{\Gamma \vdash e : A \to B \quad \Gamma \vdash e' : A}{\Gamma \vdash (e\ e') : B}\ \textbf{App}$$

$$\textbf{⇒E}\ \frac{\Gamma \vdash A \Rightarrow B \quad \Gamma \vdash A}{\Gamma \vdash A}$$

The **App** rule corresponds to modus ponens in IPL
Notice how the type is A → B but in IPL it is A ⇒ B

The **Lam** rule introduces assumptions, just as **⇒I** does in IPL

$$\frac{x \mapsto t \in \Gamma}{\Gamma \vdash x : t}\ \textbf{Var}$$

$$\textbf{Assumption}\ \frac{}{\Gamma, P \vdash P}$$

$$\frac{\Gamma \vdash e : A \to B \quad \Gamma \vdash e' : A}{\Gamma \vdash (e\ e') : B}\ \textbf{App}$$

$$\textbf{⇒E}\ \frac{\Gamma \vdash A \Rightarrow B \quad \Gamma \vdash A}{\Gamma \vdash A}$$

$$\frac{\Gamma, \{x \mapsto t\} \vdash e : A}{\Gamma \vdash (\lambda\,(x : t)\ e) : A \to B}\ \textbf{Lam}$$

$$\textbf{⇒I}\ \frac{\Gamma, A \vdash B}{\Gamma \vdash A \Rightarrow B}$$

What this means is that any time you write a proof tree in STLC, you *could have* written it in IPL instead

There is an *exact correspondence* between proof trees in IPL and STLC

This deep result means that we can execute proofs in constructive logic as programs, and that we can build programs which contain proof-relevant components (e.g., for building correct-by-construction systems).

Aside from the theoretical results, we will now use the Curry-Howard Isomorphism for a more practical reason: to add typing rules necessary for us to write more complicated programs, using forms such as pairs, etc…

We can add pairs to STLC via several additional rules, inspired by intuitionistic logic

```
e ::= (lambda (x : t) e)
    | (e e)
    | …
    | (cons e e) ;; pairs
    | (car e) | (cdr e)
```

```
t ::= int | bool | …
    | t × t ;; product types
```

The *computational* interpretation of ∧ is a pair, so we add syntax for pairs into our language

The *type* of a pair is a product type:

```
(cons 5 #t) : int × bool
```

"If e is a pair, (car/cdr e) is the type of its first/second element"

"If e_0 is type A and e_1 is type B, (cons e_0 e_1) is type A × B"

×E1 $$\frac{\Gamma \vdash e : A \times B}{\Gamma \vdash (\text{car } e) : A}$$ **×E2** $$\frac{\Gamma \vdash e : A \times B}{\Gamma \vdash (\text{cdr } e) : B}$$ **×I** $$\frac{\Gamma \vdash e_0 : A \quad \Gamma \vdash e_1 : B}{\Gamma \vdash (\text{cons } e_0\ e_1) : A \times B}$$

There are also **discriminated union** types

You may know these as C tagged **unions**

```
e ::= … ;; previous forms
    | left e
    | right e
    | case e of
        (left e0 => e0’)
        (right e1 => e1’)
```

The computational interpretation of ∨ is a *discriminated union*

```
t ::= … | t + t
```

Now we have ***sum*** types

```
(inj_left 42) : int × bool
```

Also many other types

```
(inj_left 42) : int × int
(inj_left 42) : int × (int -> int)
(inj_left 42) : int × (int × int)
```

…

A discriminated union A × B says:
"I carry *either* information of type A, *or* information of type B; but I can't promise it's exactly A or exactly B—thus, to interact with the information, you must *always* do case analysis (i.e., matching).

```
e ::= … ;; previous forms
    | left e
    | right e
    | case e of
        (left e0 => e0')
        (right e1 => e1')
```

The computational interpretation of ∨ is a *discriminated union*

```
(case (right 5) of
  (left e => e)
  (right e => 7)) ;; 7
```

```
;; In OCaml, we would write this:
# type ('a, 'b) t = Left of 'a | Right of 'b;;
type ('a, 'b) t = Left of 'a | Right of 'b
# Left (5);;
- : (int, 'a) t = Left 5
;; OCaml's type system supports general ADTs
```

Vanilla STLC

$$\frac{\Gamma \vdash e : t \to t' \quad \Gamma \vdash e' : t}{\Gamma \vdash (e\ e') : t'}\ \textbf{App} \qquad \frac{}{\Gamma \vdash n : \textbf{num}}\ \textbf{Const} \qquad \frac{x \mapsto t \in \Gamma}{\Gamma \vdash x : t}\ \textbf{Var} \qquad \frac{\Gamma, \{x \mapsto t\} \vdash e : t'}{\Gamma \vdash (\lambda\,(x : t)\ e) : t \to t'}\ \textbf{Lam}$$

Products (pairs)

$$\textbf{×E1}\ \frac{\Gamma \vdash e : A \times B}{\Gamma \vdash (\text{car}\ e) : A} \qquad \textbf{×E2}\ \frac{\Gamma \vdash e : A \times B}{\Gamma \vdash (\text{cdr}\ e) : B} \qquad \textbf{×I}\ \frac{\Gamma \vdash e_0 : A \quad \Gamma \vdash e_1 : B}{\Gamma \vdash (\text{cons}\ e_0\ e_1) : A \times B}$$

Sums (discriminated unions)

$$\textbf{+I1}\ \frac{\Gamma \vdash e : A}{\Gamma \vdash (\text{left}\ e) : A + B} \qquad \textbf{+I2}\ \frac{\Gamma \vdash e : B}{\Gamma \vdash (\text{right}\ e) : A + B} \qquad \textbf{+E}\ \frac{\Gamma \vdash e : A + B \quad \Gamma, e_0 : A \vdash e_0' : C \quad \Gamma, e_1 : A \vdash e_1' : C}{\Gamma \vdash \big(\text{case}\ e\ \text{of}\ (\text{left}\ e_0\ \Rightarrow e_0')\ (\text{right}\ e_1 \Rightarrow e_1')\big) : C}$$

Our full type system: STLC, products, unions, and negation

This type system corresponds precisely to intuitionistic logic with ⇒, ∧, ∨, and ⊥

Negation

¬A is A → ⊥

$$\frac{\Gamma \vdash e : \bot}{\Gamma \vdash (\text{case}\ e\ \text{of}) : t}$$

Completeness of STLC

- **Incomplete**: Reasonable functions we can't write in STLC
  - E.g., any program using recursion
- Several useful **extensions** to STLC
  - **Fix operator** to allow typing recursive functions
  - **Algebraic data types** to type structures
  - **Recursive types** for full algebraic data types
    - `tree = Leaf (int) | Node(int,tree,tree)`

Typing the Y Combinator

$$\frac{\Gamma \vdash f : t \to t}{\Gamma \vdash (Y f) : t}\ \mathbf{Y}$$

The “real” solution is quite nontrivial—we need *recursive types*, which may be formalized in a variety of ways

- We will not cover recursive types in this lecture, I am happy to offer pointers

Our hacky solution works in practice, but is not sound in general

- More precisely, the logic induced by the type system is no longer sound (can prove $\bot$ and therefore everything)

Typing the Y Combinator

Think of how this would look for **fib**

$$\frac{\Gamma \vdash f : t \to t}{\Gamma \vdash (Y f) : t} \; \mathbf{Y}$$

What would `t` be here?

```
(let ([fib
 (Y (λ (f) (λ (x)
             (if (= x 0)
                 1
                 (* x (fib (- x 1))))))])))
```

Error States

A program steps to an **error state** if its evaluation reaches a point where the program has not produced a value, and yet cannot make progress

```
((+ 1) (λ (x) x))
```

Gets “stuck” because + can’t operate on λ

Error States

A program steps to an **error state** if its evaluation reaches a point where the program has not produced a value, and yet cannot make progress

```
((+ 1) (λ (x) x))
```

Gets “stuck” because + can’t operate on λ

(Note that this term is **not typable** for us!)

Soundness

A type system is **sound** if no typable program
will ever evaluate to an error state

“Well typed programs cannot go wrong.”
— Milner

(You can **trust** the type checker!)

Proving Type Soundness

Theorem: if e has some type derivation, then it will evaluate to a value.

Relies on two lemmas

Progress	Preservation
If e typable, then it is either a value or can be further reduced	If e has type `t`, any reduction will result in a term of type `t`

Progress	Preservation
If `e` typable, then it is either a value or can be further reduced	If `e` has type `t`, any reduction will result in a term of type `t`

(In our system) not too hard to prove by induction on the typing derivation.

Combination of progress and preservation says: you can either take a well-typed step and maintain the invariant, or you are done (at a value).

We will skip the proof—it depends on understanding induction over derivations, chat with me if interested…

Polymorphic Type Systems

Generally also need **polymorphism** in our language. In the FP setting, we often want to talk about **parametric polymorphism**, meaning that a definition is parametric on some type variable

For example: (define (f (x : α list)) (second x)) : ∀α. α list -> α

“For all types α, this function takes an α list, and returns an α list.”

Need to introduce syntax that allows creating inductive definitions (e.g., types) which mention type variables (to enable defining them)

System F

System F extends the lambda calculus, and simply-typed lambda calculus, to enable writing functions which include lambdas ranging over **type variables**, and then to enable **computing with types**

$$\frac{\Gamma \vdash M : \forall \alpha.\, \sigma}{\Gamma \vdash M\tau : \sigma[\tau/\alpha]}\ (1) \qquad \frac{\Gamma, \alpha\ \text{type} \vdash M : \sigma}{\Gamma \vdash \Lambda\alpha.\, M : \forall \alpha.\, \sigma}\ (2)$$

The basic issue is this: now you can build nonterminating types, and thus type checking is undecidable in general

Let-Bound Polymorphism

Very common in functional languages: polymorphism but only at **binding sites** (Damas-Milner or Hindley-Milner-style polymorphism)

Decidable, relatively simple / straightforward implementation via logic programming, easy to understand

General System F unnecessary, many popular languages use HM

Type Variables...

Allows us to leave some **placeholder** variables that will be "filled in later"

```
((λ (x:t) x:t') : int -> int)
```

The int->int constraint then **forces** t = int and t' = int

Type Inference can fail...

For example, consider this case...

```
(λ (x) (λ (y:int->int) ((+ (x y)) x)))
```

No **possible** type for x! Used as fn and arg to +

Type Inference has been of interest (research and practical) for many years

It allows you to write **untyped** programs (much less painful!) and automatically *synthesize* a type for you—as long as the type **exists** (catch your mistakes)

```
(λ (f) (((f 2) 3) 4))
```


```
(λ (f : int -> int -> int -> int) (((f 2) 3) 4))
```

Type inference can be seen as einterating **all possible type assignments** to infer a valid typing. You can think of it as solving the equation:

```
∃T. (λ (f : T) (((f 2) 3) 4))
```

What is the correct type?

```
(lambda (f) (lambda (x) (if (if-zero? (f x)) 1 0)))
```

Is it:

(a) f = int->int, x = int

(b) f = bool->int, x = bool

(c) f = (int->int)->int, x = int->int

What is the correct type?

```
(lambda (f) (lambda (x) (if (if-zero? (f x)) 1 0)))
```

Is it:

(a) f = int->int, x = int

(b) f = bool->int, x = bool

(c) f = (int->int)->int, x = int->int

(d) All of the above

What is the correct type?

```
(lambda (f) (lambda (x) (if (if-zero? (f x)) 1 0)))
```

Is it:

(a) f = int->int, x = int

(b) f = bool->int, x = bool

(c) f = (int->int)->int, x = int->int

(d) All of the above

Lesson: pick a *principal* which subsumes them all, to avoid enumerating infinitely-many types.

Generalizing Type Variables...

```
(lambda (f) (lambda (x) (if (if-zero? (f x)) 1 0)))
```

Lesson:

We can't pick *just one* type. Instead, we need to be able to instantiate f and x whenever a suitable type may be found.

For example, what if we **let-bind** the lambda and use it in two different ways!?

```
(let ([g (lambda (f) (lambda (x) (if (if-zero? (f x)) 1 0)))])
  (+ ((g (lambda (x) x)) 0) ((g (lambda (x) 1)) #f)))
```

This usage requires f = nat->nat and x = nat

This usage requires f = bool->nat and x = bool

```
(lambda (f) (lambda (x) (if (if-zero? (f x)) 1 0)))
```

Instead, we can keep a generalized type by using a **type variable**, allowing a good type inference system to derive (for this example, using type var T):

Type of `f = T -> int`

Type of `x = T`

Notice that this system *demands* we must be able to compare T for equality! This is actually *nontrivial* when we add polymorphism, but is simple in STLC (structural equality)

Constraint-Based Typing

The crucial trick to implementing type inference is to use a **constraint-based** approach. In this setting, we *walk over* each subterm in the program and generate a **constraint**

Unannotated lambdas generate new **type variables**, which are later constrained by their usages

Later, we will **solve** these constraints by using a process named **unification**

Building Constraints

```
(define (build-constraints env e)
  (match e
    ;; Literals
    [(? integer? i) (cons `(,i : int) (set))]
    [(? boolean? b) (cons `(,b : bool) (set))]
    ;; Look up a type variable in an environment
    [(? symbol? x) (cons `(,x : ,(hash-ref env x)) (set))]
    ;; Lambda w/o annotation
    [`(lambda (,x) ,e)
     ;; Generate a new type variable using gensym
     ;; gensym creates a unique symbol
     (define T1 (fresh-tyvar))
     (match (build-constraints (hash-set env x T1) e)
       [(cons `(,e+ : ,T2) S)
        (cons `((lambda (,x : ,T1) ,e+) : (,T1 -> ,T2)) S)])]
    ;; Application: constrain input matches, return output
    [`(,e1 ,e2)
     (match (build-constraints env e1)
       [(cons `(,e1+ : ,T1) C1)
        (match (build-constraints env e2)
          [(cons `(,e2+ : ,T2) C2)
           (define X (fresh-tyvar))
           (cons `(((,e1+ : ,T1) (,e2+ : ,T2)) : ,X)
                 (set-union C1 C2 (set `(= ,T1 (,T2 -> ,X)))))])])]
    ;; Type stipulation against t--constrain
    [`(,e : ,t)
     (match (build-constraints env e)
       [(cons `(,e+ : ,T) C)
        (define X (fresh-tyvar))
        (cons `((,e+ : ,T) : ,X) (set-add (set-add C `(= ,X ,T)) `(= ,X ,t)))])]
    ;; If: the guard must evaluate to bool, branches must be
    ;; of equal type.
    [`(if ,e1 ,e2 ,e3)
     (match-define (cons `(,e1+ : ,T1) C1) (build-constraints env e1))
     (match-define (cons `(,e2+ : ,T2) C2) (build-constraints env e2))
     (match-define (cons `(,e3+ : ,T3) C3) (build-constraints env e3))
     (cons `((if (,e1+ : ,T1) (,e2+ : ,T2) (,e3+ : ,T3)) : ,T2)
           (set-union C1 C2 C3 (set `(= ,T1 bool) `(= ,T2 ,T3))))]))
```

Unification

At the end of constraint-building, we have a ton of equality constraints between base types and type variables

```
tv0 = int
ty1 = tv0 -> tv2
tv2 = tv3
tv3 = tv4
```

```
(lambda (x : ty1) …)
```

In this example, what is `ty1`?

Answer: think about constraints and equalities: `ty1` must be `int->int`

Unification

```
;; within the constraint constr, substitute S for T
(define (ty-subst ty X T)
  (match ty
    [(? ty-var? Y) #:when (equal? X Y) T]
    [(? ty-var? Y) Y]
    ['bool 'bool]
    ['int 'int]
    [`(,T0 -> ,T1) `(,(ty-subst T0 X T) -> ,(ty-subst T1 X T))]))

(define (unify constraints)
  ;; Substitute into a constraint
  (define (constr-subst constr S T)
    (match constr
      [`(= ,C0 ,C1) `(= ,(ty-subst C0 S T) ,(ty-subst C1 S T))]))
  ;; Is t an arrow type?
  (define (arrow? t)
    (match t [`(,_ -> ,_) #t] [_ #f]))
  ;; Walk over constraints one at a time
  (define (for-each constraints)
    (match constraints
      ['() (hash)]
      [`((= ,S ,T) . ,rest)
       (cond [(equal? S T)
              (for-each rest)]
             [(and (ty-var? S) (not (set-member? (free-type-vars T) S)))
              (hash-set (unify (map (lambda (constr) (constr-subst constr S T)) rest)) S T)]
             [(and (ty-var? T) (not (set-member? (free-type-vars S) T)))
              (hash-set (unify (map (lambda (constr) (constr-subst constr T S)) rest)) T S)]
             [(and (arrow? S) (arrow? T))
              (match-define `(,S1 -> ,S2) S)
              (match-define `(,T1 -> ,T2) T)
              (unify (cons `(= ,S1 ,T1) (cons `(= ,S2 ,T2) rest)))]
             [else (error "type failure")])]))
```

Why Type Theory?

Why is type inference / checking useful?

- Can write **fully-verified** programs.
  - Cons: type systems are esoteric, complicated, academic, etc…
    - Popular languages (Swift, Rust, etc…) are *tending towards more elaborate type systems as they evolve*

- Type synthesis offers me “proofs for free:”
  - “If my program type checks it works” — **not** true in C/C++/…

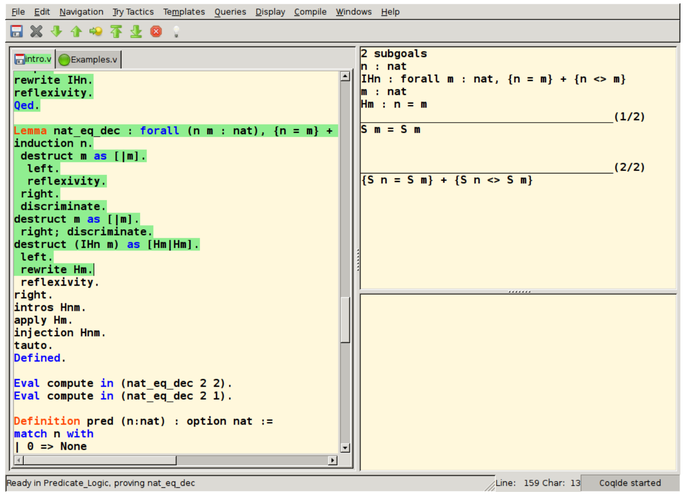
- Less **mental burden**, like CoPilot (etc… tools), type systems can integrate into IDEs to use synthesis information in guiding programming
  - In some ways, this reflects the logical statements underlying the type system’s design (Curry Howard)

“Proofs as Programs”

A significant amount of interest has been given to programming languages which use **powerful type systems** to write programs *alongside a proof of the program’s correctness*

Imagine how nice it would be to write a **completely-formally-verified** program—no bugs ever again!


```
"prove(M,I) :- append(Q,[C|R],M), \+member(-_,C),
  append(Q,R,S), prove([!],[[-!|C]|S],[],I).
 prove([],_,_,_).
 prove([L|C],M,P,I) :- (-N=L; -L=N) -> (member(N,P);
  append(Q,[D|R],M), copy_term(D,E), append(A,[N|B],E),
  append(A,B,F), (D==E -> append(R,Q,S); length(P,K), K<I,
  append(R,[D|Q],S)), prove(F,S,[L|P],I)), prove(C,M,P,I)."
```



Dependent Type Systems

We can construct type systems / PL where terms can have output types depend on input terms:

```
∀ (l : list A) : {l' : sorted l' ∧ ∀ (x : A). (member l x) ⇒ (member l' x)}
```

These are called *dependent types*, because types can depend on *values*

- This allows expressing that l' is sorted
- Unfortunately, these type systems are *significantly* more complicated (need to write proofs)
- Worse, even type *checking* may be **undecidable** (in general)

Precise formalization of these systems is beyond the scope of this class

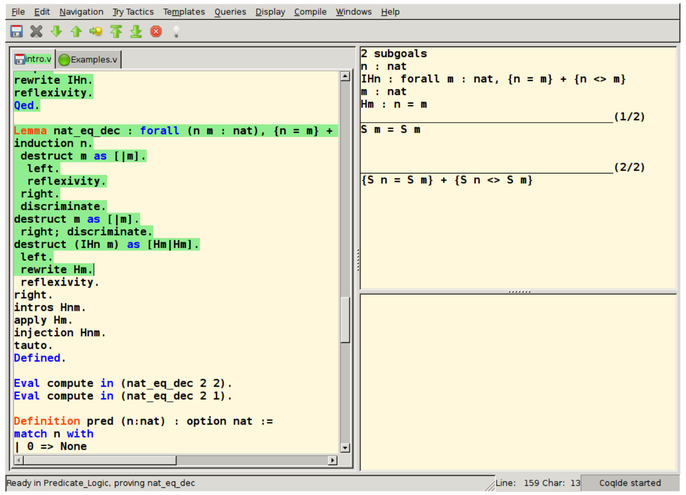
A huge family of languages have popped up to implement dependent type systems and subsequently enable “fully-verified” programming

They hit a variety of expressivity points. The fundamental trade off is: (a) expressivity vs. (b) automation.

Highly-expressive systems require you to write all the proofs yourself, and a lot of manual annotation (potentially).

Agda

```
"prove(M,I) :- append(Q,[C|R],M), \+member(-_,C),
  append(Q,R,S), prove([!],[[-!|C]|S],[],I).
 prove([],_,_,_).
 prove([L|C],M,P,I) :- (-N=L; -L=N) -> (member(N,P);
  append(Q,[D|R],M), copy_term(D,E), append(A,[N|B],E),
  append(A,B,F), (D==E -> append(R,Q,S); length(P,K), K<I,
  append(R,[D|Q],S)), prove(F,S,[L|P],I)), prove(C,M,P,I)."
```



Wrap-Up: Type Systems

- **Avoid unnecessary overhead to ensure safety**
  - Generally need ***some*** runtime overhead to ensure safety
    - Hard to beat expertly-written C code that uses zero abstractions
  - Thus, types can help us ***optimize*** program by avoiding unnecessary checks
    - Where / when is typing done? How does the compiler leverage it?

  - **Ensure mathematically-pleasing properties**
    - Want to know our program is free from bugs / type errors — if PL has a sound type system, then we know type correctness = no runtime type errors
      - But other kinds of errors (bad array index, division by zero, etc.) may still remain
    - Proving ***total*** correctness ***is*** possible, but only with advanced type systems
      - E.g., the “Lean” programming language / theorem prover (mathlib, Tao, etc.)

- Today: most truly safe languages generally have ***some*** runtime overhead for safety, and ***some*** opportunity for runtime exceptions in practice. But Rust is getting ***very*** close (or better!) w/ types!